

08-first-steps-road-to-the-top-part-1

May 30, 2025

```
[3]: # install fastkaggle if not available
try: import fastkaggle
except ModuleNotFoundError:
    !pip install -Uq fastkaggle

from fastkaggle import *
```

In [Iterate Like a Grandmaster](#) I explained that when working on a Kaggle project:

...the focus generally should be two things:

1. Creating an effective validation set
2. Iterating rapidly to find changes which improve results on the validation set.

Here I'm going to go further, showing the process I used to tackle the [Paddy Doctor](#) competition, leading to four submissions in a row which all were (at the time of submission) in 1st place, each one more accurate than the last. You might be surprised to discover that the process of doing this was nearly entirely mechanistic and didn't involve any consideration of the actual data or evaluation details at all.

This notebook is the first in a series showing every step of the process. At the end of this notebook we'll have a basic submission; by the end of the series you'll see how I got to the top of the table!

As a special extra, I'm also opening up early a selection of "walkthru" videos that we've been preparing for the new upcoming fast.ai course. Each day I do a walkthru with fast.ai fellows and registered students, and we record those sessions. They'll all be released at the same time as the next course (probably August 2022), but I'm releasing the ones covering this competition right now! Here they are:

- [Walkthru 8](#)
- [Walkthru 9](#)
- [Walkthru 10](#)
- [Walkthru 11](#)
- [Walkthru 12](#)
- [Walkthru 13](#)

0.1 Getting set up

First, we'll get the data. I've just created a new library called [fastkaggle](#) which has a few handy features, including getting the data for a competition correctly regardless of whether we're running

on Kaggle or elsewhere. Note you'll need to first accept the competition rules and join the competition, and you'll need your kaggle API key file `kaggle.json` downloaded if you're running this somewhere other than on Kaggle. `setup_comp` is the function we use in `fastkaggle` to grab the data, and install or upgrade our needed python modules when we're running on Kaggle:

```
[4]: comp = 'paddy-disease-classification'

path = setup_comp(comp, install='fastai "timm>=0.6.2.dev0"')
```

Downloading paddy-disease-classification.zip to /workspaces/elec4630-paddy-deepfake-classification

100%| | 1.02G/1.02G [00:49<00:00, 22.0MB/s]

```
[5]: path
```

```
[5]: Path('paddy-disease-classification')
```

Now we can import the stuff we'll need from `fastai`, set a seed (for reproducibility – just for the purposes of making this notebook easier to write; I don't recommend doing that in your own analysis however) and check what's in the data:

```
[6]: from fastai.vision.all import *
set_seed(42)

path.ls()
```

```
[6]: (#4) [Path('paddy-disease-classification/test_images'),Path('paddy-disease-
classification/train.csv'),Path('paddy-disease-
classification/sample_submission.csv'),Path('paddy-disease-
classification/train_images')]
```

0.2 Looking at the data

The images are in `train_images`, so let's grab a list of all of them:

```
[7]: trn_path = path/'train_images'
files = get_image_files(trn_path)
```

...and take a look at one:

```
[8]: img = PILImage.create(files[0])
print(img.size)
img.to_thumb(128)
```

(480, 640)

```
[8]:
```



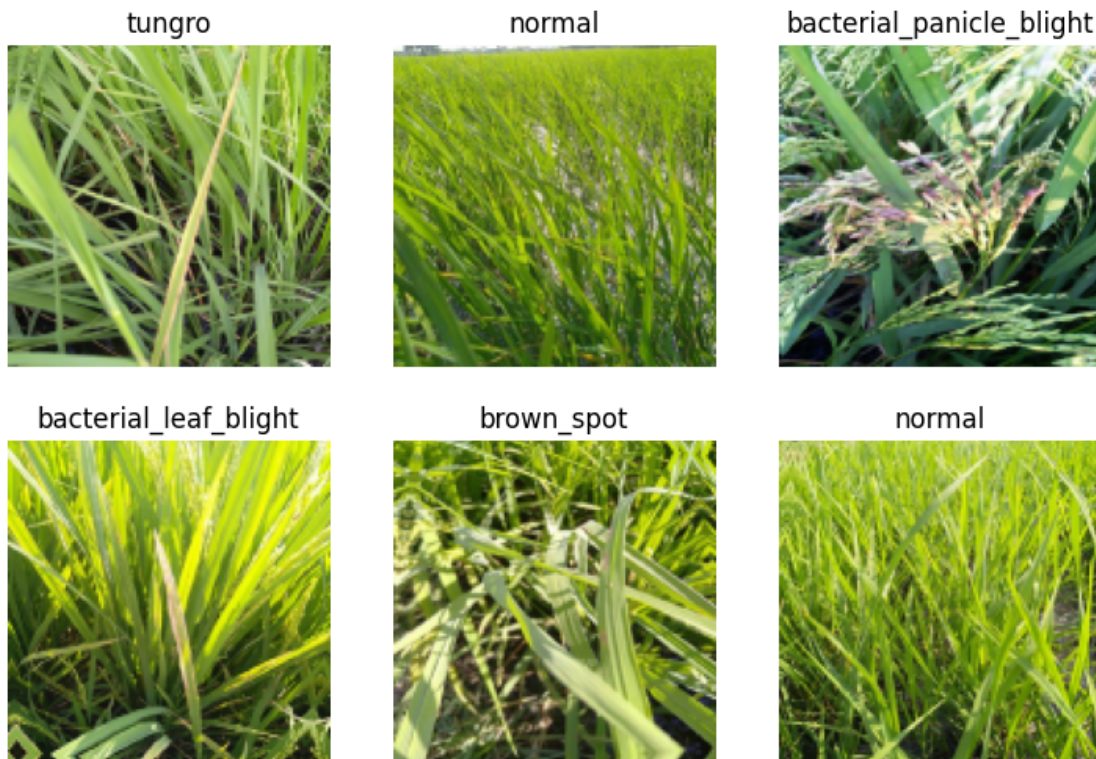
Looks like the images might be 480x640 – let’s check all their sizes. This is faster if we do it in parallel, so we’ll use fastcore’s `parallel` for this:

```
[9]: from fastcore.parallel import *  
  
def f(o): return PILImage.create(o).size  
sizes = parallel(f, files, n_workers=8)  
pd.Series(sizes).value_counts()
```

```
[9]: (480, 640)    10403  
     (640, 480)         4  
     Name: count, dtype: int64
```

They’re nearly all the same size, except for a few. Because of those few, however, we’ll need to make sure we always resize each image to common dimensions first, otherwise fastai won’t be able to create batches. For now, we’ll just squish them to 480x480 images, and then once they’re in batches we do a random resized crop down to a smaller size, along with the other default fastai augmentations provided by `aug_transforms`. We’ll start out with small resized images, since we want to be able to iterate quickly:

```
[10]: dls = ImageDataLoaders.from_folder(trn_path, valid_pct=0.2, seed=42,  
      item_tfms=Resize(480, method='squish'),  
      batch_tfms=aug_transforms(size=128, min_scale=0.75))  
  
dls.show_batch(max_n=6)
```



0.3 Our first model

Let's create a model. To pick an architecture, we should look at the options in [The best vision models for fine-tuning](#). I like the looks of `resnet26d`, which is the fastest resolution-independent model which gets into the top-15 lists there.

```
[11]: learn = vision_learner(dls, 'resnet26d', metrics=error_rate, path='.').to_fp16()
```

```
model.safetensors: 0%|          | 0.00/64.2M [00:00<?, ?B/s]
```

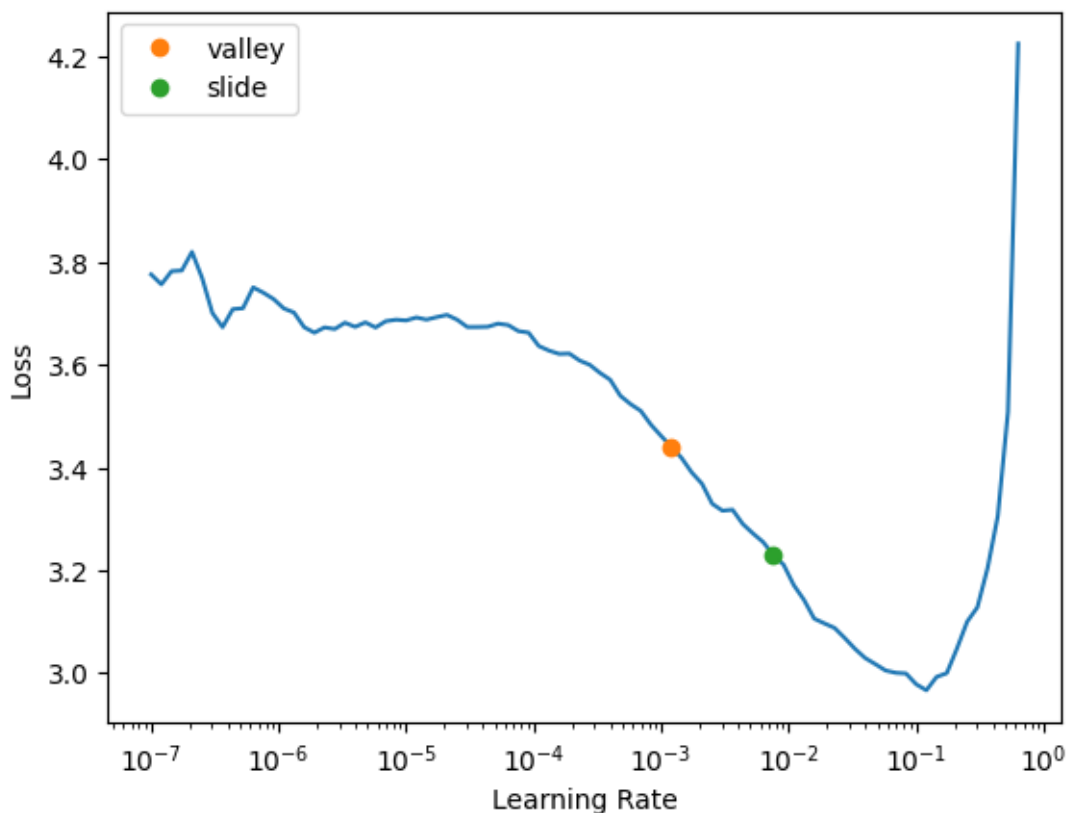
Let's see what the learning rate finder shows:

```
[12]: learn.lr_find(suggest_funcs=(valley, slide))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[12]: SuggestedLRs(valley=0.0012022644514217973, slide=0.007585775572806597)
```



`lr_find` generally recommends rather conservative learning rates, to ensure that your model will train successfully. I generally like to push it a bit higher if I can. Let's train a few epochs and see how it looks:

```
[13]: learn.fine_tune(3, 0.01)
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

We're now ready to build our first submission. Let's take a look at the sample Kaggle provided to see what it needs to look like:

0.4 Submitting to Kaggle

```
[14]: ss = pd.read_csv(path/'sample_submission.csv')
      ss
```

```
[14]:      image_id  label
      0      200001.jpg   NaN
      1      200002.jpg   NaN
      2      200003.jpg   NaN
      3      200004.jpg   NaN
      4      200005.jpg   NaN
      ...      ...      ...
    3464    203465.jpg   NaN
    3465    203466.jpg   NaN
    3466    203467.jpg   NaN
    3467    203468.jpg   NaN
    3468    203469.jpg   NaN
```

```
[3469 rows x 2 columns]
```

OK so we need a CSV containing all the test images, in alphabetical order, and the predicted label for each one. We can create the needed test set using fastai like so:

```
[15]: tst_files = get_image_files(path/'test_images').sorted()
      tst_dl = dls.test_dl(tst_files)
```

We can now get the probabilities of each class, and the index of the most likely class, from this test set (the 2nd thing returned by `get_preds` are the targets, which are blank for a test set, so we discard them):

```
[16]: probs,_,idxs = learn.get_preds(dl=tst_dl, with_decoded=True)
      idxs
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
[16]: tensor([7, 8, 4, ..., 8, 1, 5])
```

These need to be mapped to the names of each of these diseases, these names are stored by fastai automatically in the vocab:

```
[17]: dls.vocab
```

```
[17]: ['bacterial_leaf_blight', 'bacterial_leaf_streak', 'bacterial_panicle_blight',
      'blast', 'brown_spot', 'dead_heart', 'downy_mildew', 'hispa', 'normal',
      'tungro']
```

We can create an apply this mapping using pandas:

```
[18]: mapping = dict(enumerate(dls.vocab))
      results = pd.Series(idxs.numpy(), name="idxs").map(mapping)
      results
```

```
[18]: 0          hispa
      1          normal
      2      brown_spot
      3          blast
      4          blast
      ...
      3464      dead_heart
      3465          hispa
      3466          normal
      3467      bacterial_leaf_streak
      3468      dead_heart
      Name: idxs, Length: 3469, dtype: object
```

Kaggle expects the submission as a CSV file, so let's save it, and check the first few lines:

```
[19]: ss['label'] = results
      ss.to_csv('subm.csv', index=False)
      !head subm.csv
```

```
image_id,label
200001.jpg,hispa
200002.jpg,normal
200003.jpg,brown_spot
200004.jpg,blast
200005.jpg,blast
200006.jpg,brown_spot
200007.jpg,dead_heart
200008.jpg,brown_spot
200009.jpg,normal
```

Let's submit this to kaggle. We can do it from the notebook if we're running on Kaggle, otherwise we can use the API:

```
[20]: if not iskaggle:
      from kaggle import api
      api.competition_submit_cli('subm.csv', 'initial rn26d 128px', comp)
```

Warning: Looks like you're using an outdated API Version, please consider updating (server 1.7.4.2 / client 1.5.16)

```
100%|          | 69.9k/69.9k [00:01<00:00, 63.8kB/s]
```

Success! We successfully created a submission.

0.5 Conclusion

Our initial submission is not very good (top 80% of teams) but it only took a minute to train. The important thing is that we have a good starting point to iterate from, and we can do rapid iterations. Every step from loading the data to creating the model to submitting to Kaggle is all automated and runs quickly.

Therefore, we can now try lots of things quickly and easily and use those experiments to improve our results. In the next notebook, we'll do exactly that!

If you found this notebook useful, please remember to click the little up-arrow at the top to upvote it, since I like to know when people have found my work useful, and it helps others find it too. And if you have any questions or comments, please pop them below – I read every comment I receive!

0.6 Addendum

`fastkaggle` also provides a function that pushes a notebook to Kaggle Notebooks. I wrote this notebook on my own machine, and pushed it to Kaggle from there – here's the command I used:

```
[22]: if not iskaggle:
        push_notebook('isaacjohanziebarth', 'first-steps-road-to-the-top-part-1',
                        title='First Steps: Road to the Top, Part 1',
                        file='08-first-steps-road-to-the-top-part-1.ipynb',
                        competition=comp, private=False, gpu=True)
```

Kernel version 2 successfully pushed. Please check progress at
<https://www.kaggle.com/code/isaacjohanziebarth/first-steps-road-to-the-top-part-1>

```
[ ]:
```